CS604 COMPUTER VISION · TEAM 06

Presentation Script

Slides 1–14 of `cs604_team06_slides_v2.pptx` · 7 November 2026

One page per slide. Each page shows the slide, who speaks, about how long, the words to say, and short notes on the hard terms. Read the script in your own voice; you do not need to memorize every word. Lines marked *If asked* in the notes are not part of the talk; keep them for questions.

Slide	Title	Speaker	Time
1	Prototype-initialized adaptation of ViT-B/16 for 10,000-class fine-grained classification	A	about 35 s
2	Many classes, few photos, one budget	A	about 60 s
3	Baselines first, then seven methods	A	about 60 s
4	How we iterated: one question at a time	B	about 60 s
5	Spread small changes over every block	B	about 70 s
6	Linear-probe heads start high, finish low	B	about 55 s
7	Our fix: start from class prototypes	C	about 70 s
8	Lower training loss, yet lower accuracy	C	about 70 s
9	Each change, tested one at a time	C	about 60 s
10	Full fine-tuning trains every weight	D	about 50 s
11	LoRA trains 1.38%, then merges	D	about 45 s
12	Final results on every metric	D	about 50 s
13	Input resolution: the largest gain we found	D	about 55 s
14	Takeaways	D	about 55 s

Total: 1732 words, about 13:19 at 130 words per minute (about 14:26 at a slower 120). By speaker: A: 2:40 B: 3:05 C: 3:22 D: 4:13. Slide 15 (Thank you) and the questions come after this.

Before the talk: replace [Name B], [Name C] and [Name D] in the handoff lines (and on slides 1 and 15) with your real names. Speak slowly, look at the audience, and pause after each key number.

SLIDE 1 · SPEAKER A · ABOUT 35 SECONDS

Prototype-initialized adaptation of ViT-B/16 for 10,000-class fine-grained classification



Good afternoon, everyone. We are Team 06.

We started from ViT-B/16. It is a Vision Transformer that already knows how to recognize images. Our job was to teach it 10,000 new classes that look very similar, within a fixed compute budget.

We compared seven ways to adapt the model. One choice mattered more than the method itself: how the new classification head starts.

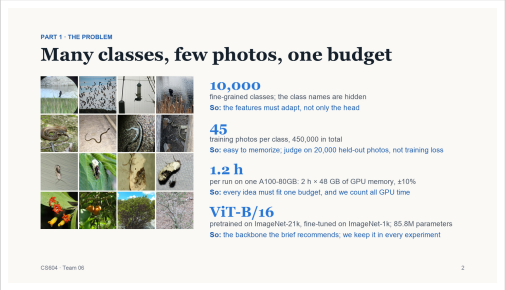
On the hidden test set, our full fine-tuning model reaches 76.09%. Our LoRA model reaches 71.67%.

Notes for difficult terms

- **Pretrained model:** a model that was already trained on a large image dataset.
- **Head:** the last layer of the model. It gives one score for each class.
- **Fine-grained:** the classes look very alike, like different kinds of birds.

SLIDE 2 · SPEAKER A · ABOUT 60 SECONDS

Many classes, few photos, one budget



So, what makes this problem hard? We found four things. Each one led to a design choice.

One: 10,000 fine-grained classes, with hidden names. The pretrained model never learned such small differences. So the whole model has to adapt, not just the last layer.

Two: only 45 training photos per class, and they look very different. On the left, one class can be a single bird or a whole flock. With so few photos, a model can just memorize them. So we never trust the training loss. We keep 2 photos per class aside, 20,000 in total, and test on them. This is our holdout.

Three: a budget of 1.2 GPU-hours per run. Every idea had to fit in it.

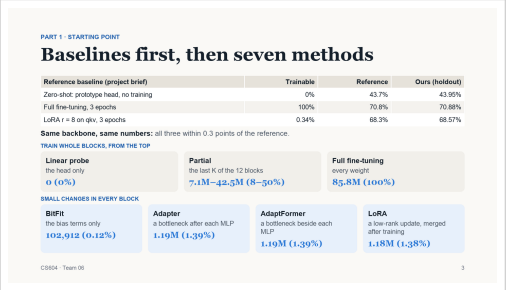
Four: we kept ViT-B/16, the backbone the brief recommends, in every experiment.

Notes for difficult terms

- **Training loss:** how wrong the model is on its own training photos. It can look good even when the model only memorizes.
- **Holdout:** training photos we set aside and never train on. We use them like a practice exam.
- **Budget:** 2 hours × 48 GB of GPU memory. On our 80 GB GPU, that is 96 / 80 = 1.2 hours.
- **If asked:** our holdout scores match the official CodaBench scores within about 0.3 points.

SLIDE 3 · SPEAKER A · ABOUT 60 SECONDS

Baselines first, then seven methods



Before trying anything new, we checked our code. We reproduced the three reference results from the brief, all within 0.3 points. For example, full fine-tuning: 70.8% in the brief, and 70.88% for us.

Then we compared seven methods, in two groups.

The first group trains whole blocks, from the top. The linear probe trains only the head. Partial fine-tuning trains the last few blocks. Full fine-tuning trains everything.

The second group makes small changes in every block. BitFit trains only the biases. Adapter and AdaptFormer add a small module to each block. LoRA learns a small update and merges it in after training. This group trains only 0.12 to 1.4% of the model.

All seven use the same settings, so the comparison is fair.

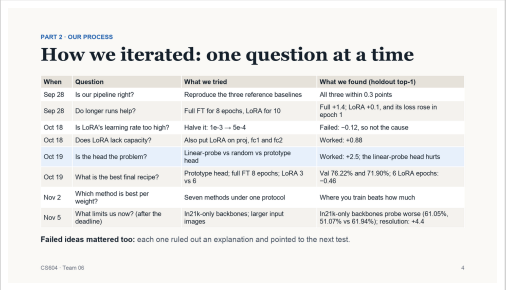
Now **[Name B]** will show how we worked through these experiments.

Notes for difficult terms

- **Baseline:** a simple reference result to compare against.
- **Block:** ViT has 12 identical blocks, stacked one after another.
- **Bias:** one small extra number per output in each layer.

SLIDE 4 · SPEAKER B · ABOUT 60 SECONDS

How we iterated: one question at a time



PART 2 · OUR PROCESS

How we iterated: one question at a time

When	Question	What we tried	What we found (holdout top-1)
Sep 28	Is our pipeline right?	Reproduce the three reference baselines	All three within 0.3 points
Sep 28	Do longer runs help?	Full FT for 8 epochs, LoRA for 10	Full +1.4; LoRA +0.1, and its loss rose in epoch 1
Oct 18	Is LoRA's learning rate too high?	Halve it: 1e-3 → 5e-4	Failed: −0.12, so not the cause
Oct 18	Does LoRA lack capacity?	Also put LoRA on proj, fc1 and fc2	Worked: +0.88
Oct 19	Is the head the problem?	Linear-probe vs random vs prototype head	Worked: +2.5; the linear-probe head hurts
Oct 19	What is the best final recipe?	Prototype head; full FT 8 epochs; LoRA 3 vs 6	Val 76.22% and 71.90%; 6 LoRA epochs: −0.46
Nov 2	Which method is best per weight?	Seven methods under one protocol	Where you train beats how much
Nov 5	What limits us now? (after the deadline)	In21k-only backbones; larger input images	In21k-only backbones probe worse (61.05%, 51.07% vs 61.94%); resolution: +4.4

Failed ideas mattered too: each one ruled out an explanation and pointed to the next test.

CS604 · Team 06 4

Thank you. This table shows our process: one question per round. Each answer told us what to try next.

First, we trained longer. Full fine-tuning gained 1.4 points. LoRA barely moved, and its loss even went up in the first epoch.

Our first guess was that LoRA's learning rate was too high. We halved it. Nothing changed, so that guess was wrong.

Our second guess was too little capacity. We added LoRA to more layers and gained almost one point.

Then we asked: is the head the problem? That was the big one: plus 2.5 points. More on that soon.

After the deadline, we asked what still limits us. It was not the backbone. It was the image size.

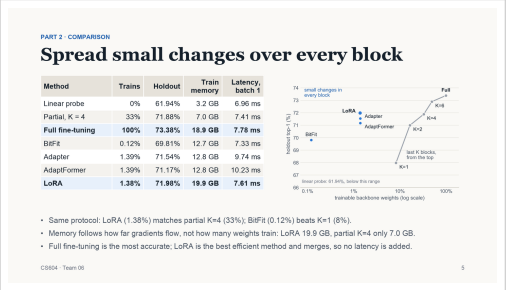
So even failed ideas helped. Each one ruled out a wrong explanation.

Notes for difficult terms

- **Epoch:** one full pass over all the training photos.
- **Learning rate:** how big each training step is.
- **Capacity:** how much the model is able to change and learn.

SLIDE 5 · SPEAKER B · ABOUT 70 SECONDS

Spread small changes over every block


PART 2 · COMPARISON

Spread small changes over every block

Method	Trains	Holdout	Train memory	Latency, batch 1
Linear probe	0%	61.94%	3.2 GB	6.96 ms
Partial, K = 4	33%	71.88%	7.0 GB	7.41 ms
Full fine-tuning	**100%**	**73.38%**	**18.9 GB**	**7.78 ms**
BitFit	0.12%	69.81%	12.7 GB	7.33 ms
Adapter	1.39%	71.54%	12.8 GB	9.74 ms
AdaptFormer	1.39%	71.17%	12.8 GB	10.23 ms
LoRA	**1.38%**	**71.98%**	**19.9 GB**	**7.61 ms**

- Same protocol: LoRA (1.38%) matches partial K=4 (33%); BitFit (0.12%) beats K=1 (8%).
- Memory follows how far gradients flow, not how many weights train: LoRA 19.9 GB, partial K=4 only 7.0 GB.
- Full fine-tuning is the most accurate; LoRA is the best efficient method and merges, so no latency is added.

CS604 · Team 06 5

Here are all seven methods. The plot shows accuracy against the share of weights we train, on a log scale.

The grey line trains whole blocks from the top. One block, 8% of the weights, gives 68%.

The blue points make small changes in every block. They sit far to the left. LoRA trains just 1.38% of the weights, and it matches training the last four blocks, which is 33%. BitFit trains only the biases, and still beats training the last block.

So where you train matters more than how much.

Memory tells the same story. LoRA needs 19.9 GB, more than full fine-tuning. The last four blocks need only 7 GB. Memory depends on how far back the gradients must flow.

Full fine-tuning is still the most accurate. Among the efficient methods, LoRA is the best. It merges into the weights, so it adds no delay.

Notes for difficult terms

- **Log scale:** each step on the axis is ten times bigger: 0.1%, 1%, 10%, 100%.
- **Gradient:** the signal that tells each weight how to change. To compute it, the model keeps in-between results back to the first layer it trains. That costs memory.
- **Merge:** LoRA's learned change is added into the original weights, so the model keeps the same size and speed.
- **If asked:** the two adapters cannot merge, so they add 2 to 3 milliseconds per photo.

SLIDE 6 · SPEAKER B · ABOUT 55 SECONDS

Linear-probe heads start high, finish low



Now, the head problem. Our first runs used a popular recipe called LP-FT, from Kumar and colleagues. First, train a new head on the frozen model. That is a linear probe. Then, fine-tune everything.

The leftmost point in each chart is the head alone, before training. The linear-probe head starts at 62%. The prototype head starts at only 44%.

But after three epochs, the linear-probe head is about 2.5 points behind, for both full fine-tuning and LoRA. It starts high, but finishes low.

We first suspected LoRA's learning rate, but halving it changed nothing. So we tested the head. A prototype head, and even a random head, beat the linear-probe head by about 2.5 points.

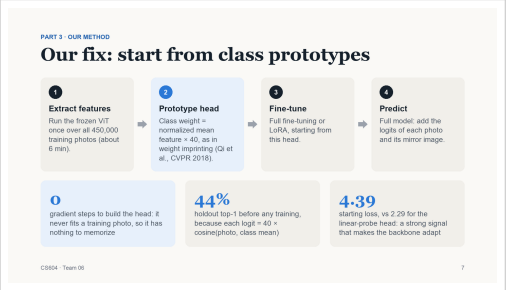
[Name C] will now explain our fix.

Notes for difficult terms

- **Frozen:** the weights do not change during training.
- **Linear probe:** train only a new last layer on top of a frozen model.
- **LP-FT:** Linear Probe, then Fine-Tune.

SLIDE 7 · SPEAKER C · ABOUT 70 SECONDS

Our fix: start from class prototypes



Thanks. Our fix is simple: start the head from class prototypes, in four steps.

Step one: run the frozen model once over all 450,000 training photos. Each photo becomes 768 numbers, called a feature.

Step two is the key. For each class, we average the direction of its features. That is the class prototype. It becomes the weight for that class, times 40. This idea, weight imprinting, comes from Qi and colleagues. What is new is our finding: here, it beats the usual linear-probe start.

Step three: fine-tune from this head, with full fine-tuning or LoRA.

Step four: the full model also scores the mirror image of each photo, and adds the two scores.

Why is this a good start? It takes zero training steps, so it has nothing to memorize. Yet it already gets 44% before training. And its starting loss is higher, 4.39 against 2.29. That gives the model a strong signal to learn from.

Notes for difficult terms

- **Feature:** the 768 numbers the model uses to describe a photo.
- **Prototype:** the typical feature of a class, found by averaging.
- **Times 40:** makes the scores more spread out. Each score is 40 times the cosine similarity between the photo and the class prototype.
- **Cosine similarity:** how much two directions point the same way, from -1 to 1.

SLIDE 8 · SPEAKER C · ABOUT 70 SECONDS

Lower training loss, yet lower accuracy



Here is the evidence: two full fine-tuning runs, 8 epochs on all data. The only difference is the head.

The orange run uses the linear-probe head. Its training loss is lower the whole time: 1.77 against 1.85 at the end. But on validation, it scores 73.10%, against 76.22%. Lower training loss, but lower accuracy. That is memorization.

Why does it happen? One: the linear-probe head was trained on the same training photos. Two: so its starting loss is low, and the learning signal is weak. When the features change, the head no longer fits them. That is why LoRA's loss went up at first. Three: the model ends up memorizing the 45 photos per class.

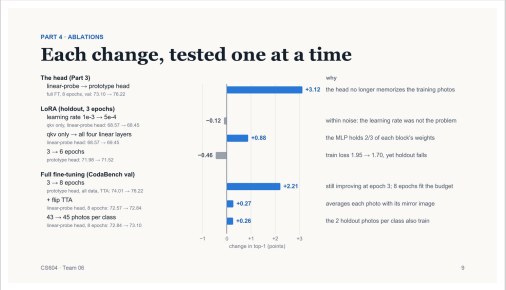
We also ran a control. A random head ends level with the prototype head: 71.95 against 72.00. So prototypes are not magic. The harm comes from fitting the head on the training photos.

Notes for difficult terms

- **Validation set:** official photos with hidden labels, scored on the CodaBench website.
- **Memorization (overfitting):** doing well on the training photos, but worse on new photos.
- **Control:** an extra test that rules out another explanation.
- The slide shows LoRA's first-epoch loss rising from 2.2 to 2.7.

SLIDE 9 · SPEAKER C · ABOUT 60 SECONDS

Each change, tested one at a time



This chart shows why each setting in our recipe is there. Each row changes one thing and keeps the rest the same.

The head is the biggest change: plus 3.1 points.

For LoRA, halving the learning rate did nothing. Putting LoRA on all four linear layers gave about 0.9 points. That is because two thirds of each block's weights are in the MLP.

Training LoRA longer hurt. Its training loss kept falling, but the holdout score fell too. That is overfitting, so we stop LoRA at 3 epochs.

Full fine-tuning is the opposite. Going from 3 to 8 epochs added 2.2 points, and it still fit the budget.

The mirror-image trick and all 45 photos per class add about a quarter point each.

Now **[Name D]** will show our two final models.

Notes for difficult terms

- **Ablation:** changing one part at a time to see how much it matters.
- **The four linear layers in a block:** qkv and proj in attention, fc1 and fc2 in the MLP.
- **Mirror-image trick:** flip test-time augmentation (flip TTA).

SLIDE 10 · SPEAKER D · ABOUT 50 SECONDS

Full fine-tuning trains every weight



Thank you. Here are our two final models. Blue means trained, and grey means frozen.

The first is full fine-tuning, so everything is blue. We train all 85.8 million weights, plus the new head with 7.69 million weights. The head starts from the class prototypes.

We use cross-entropy loss with label smoothing of 0.1, and the AdamW optimizer. The learning rate is 1e-4, and it shrinks toward the input. Each block gets 0.75 times the rate of the block above it. So the first block changes very little. We also use drop path against overfitting.

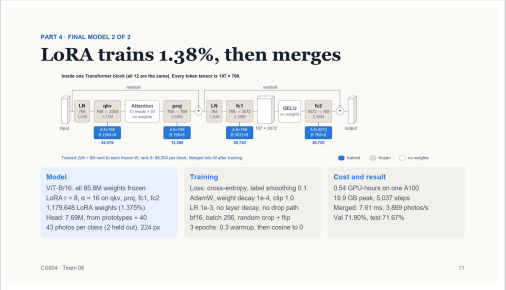
We train 8 epochs on all 45 photos per class, in 1.03 GPU-hours.

Notes for difficult terms

- **Cross-entropy:** the loss that checks how much probability the model gives to the right class.
- **Label smoothing 0.1:** the target for the right class is about 0.9 instead of 1, so the model is not pushed to be overconfident.
- **AdamW:** a common optimizer. The W stands for weight decay, which gently pulls weights toward zero.
- **Drop path:** randomly skips some blocks during training, so the model does not rely too much on one part.
- The first block gets only about 3% of the learning rate (0.75 to the power of 12).

SLIDE 11 · SPEAKER D · ABOUT 45 SECONDS

LoRA trains 1.38%, then merges



The second model is LoRA, shown inside one block. The original weights stay frozen.

Next to four layers in each block, qkv, proj, fc1 and fc2, we add two small matrices, A and B. Only they learn, plus the head. That is 1.18 million weights, or 1.375% of the model.

The learning rate is 1e-3. We train only 3 epochs, because longer training overfits. It took 0.54 GPU-hours.

And here is the nice part. After training, we merge A and B back into the weights. So the final model has the same shape and the same speed as before.

Notes for difficult terms

- **Rank r = 8:** the thin side of A and B. The learned change is B times A.
- **Alpha = 16:** a scale. The change is multiplied by alpha divided by r, which is 2.
- **Merge:** new weight = old weight + 2 × B × A.

SLIDE 12 · SPEAKER D · ABOUT 50 SECONDS

Final results on every metric


PART 4 · FINAL MODELS

Final results on every metric

Model	Trainable	Val	Test	GPU-hours	Peak memory	Latency, batch 1, A100
Project brief reference: full fine-tuning	100%	70.8%	–	–	–	–
Ours: full FT, prototype head, 8 epochs, all data, flip TTA	**85.8M (100%)**	**76.22%**	**76.09%**	**1.03**	**18.9 GB**	**14.95 ms (2 passes)**
Project brief reference: LoRA r = 8 on qkv	0.34%	68.3%	–	–	–	–
Ours: LoRA on all linear layers, prototype head, 3 epochs	**1.18M (1.375%)**	**71.90%**	**71.67%**	**0.54**	**19.9 GB**	**7.61 ms (merged)**

- **Held up on test:** within 0.25 points of val for both models.
- **Inside the rules:** both fit the 1.2-hour budget; both test files come from our Phase 1 models.
- **Recipe:** AdamW, cosine, bf16, batch 256; full FT at LR 1e-4, layer decay 0.75; LoRA r = 8 at 1e-3.
- **Fair comparison:** the like-for-like test is slide 5; here each model got its best setting.

CS604 · Team 06 12

Here are the results on every metric the brief asks for.

Full fine-tuning: 76.22% on validation and 76.09% on the hidden test set. That is 5.4 points above the reference, in 1.03 GPU-hours.

LoRA trains only 1.375% of the model. It gets 71.90% on validation and 71.67% on test. That is 3.6 points above the reference LoRA, in 0.54 GPU-hours.

For both models, test is within 0.25 points of validation. Both fit the budget. And both test files come from our Phase 1 models, as the rules require.

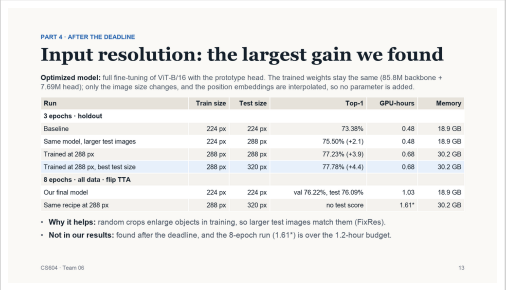
One note: here each model uses its own best setting. The like-for-like comparison is on slide 5.

Notes for difficult terms

- **Validation and test:** two sets with hidden labels. The test set decides the final ranking.
- **Latency:** full fine-tuning takes 14.95 ms per photo, because the mirror-image trick runs the model twice. Merged LoRA takes 7.61 ms.
- **Like-for-like (slide 5, same settings):** 73.38% for full fine-tuning against 71.98% for LoRA.

SLIDE 13 · SPEAKER D · ABOUT 55 SECONDS

Input resolution: the largest gain we found



PART 4 · AFTER THE DEADLINE

Input resolution: the largest gain we found

Optimized model: full fine-tuning of ViT-B/16 with the prototype head. The trained weights stay the same (85.8M backbone + 7.69M head); only the image size changes, and the position embeddings are interpolated, so no parameter is added.

Run	Train size	Test size	Top-1	GPU-hours	Memory
3 epochs · holdout					
Baseline	224 px	224 px	73.38%	0.48	18.9 GB
Same model, larger test images	224 px	288 px	75.50% (+2.1)	0.48	18.9 GB
Trained at 288 px	288 px	288 px	77.23% (+3.9)	0.68	30.2 GB
Trained at 288 px, best test size	288 px	320 px	77.78% (+4.4)	0.68	30.2 GB
8 epochs · all data · flip TTA					
Our final model	224 px	224 px	val 76.22%, test 76.09%	1.03	18.9 GB
Same recipe at 288 px	288 px	320 px	no test score	1.61*	30.2 GB

- **Why it helps:** random crops enlarge objects in training, so larger test images match them (FixRes).
- **Not in our results:** found after the deadline, and the 8-epoch run (1.61*) is over the 1.2-hour budget.

CS604 · Team 06 13

One more lesson, from after the deadline. Same full fine-tuning model, same weights. We only changed the image size.

With our short 3-epoch setup, simply testing on larger images adds 2.1 points. That is 288 pixels instead of 224. Training at 288 and testing at 320 gives 77.78%, 4.4 points above the baseline. That still fits the budget, but it needs 30 GB of memory instead of 19.

Why does it help? Random crops in training zoom in, so objects look bigger. Larger test images match that. This is called FixRes.

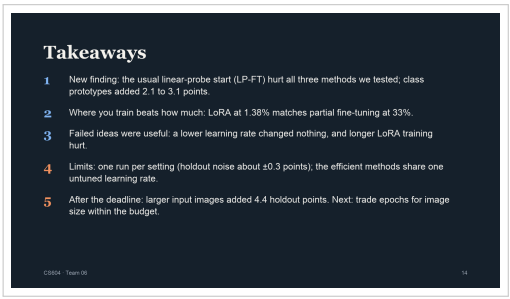
But it is not part of our results. We found it after the deadline, and our full 8-epoch recipe at 288 pixels would be over budget. So it has no test score.

Notes for difficult terms

- **Resolution:** the image size in pixels. More pixels mean more patches (197 tokens at 224 pixels, 325 at 288), so more compute and memory.
- **Position embeddings:** tell the model where each patch is. For larger images they are stretched to the new grid, so no new parameters are added.
- The full 8-epoch recipe at 288 pixels would need 1.61 GPU-hours; the budget is 1.2.

SLIDE 14 · SPEAKER D · ABOUT 55 SECONDS

Takeaways



To sum up, five points.

One: the usual linear-probe start hurt all three methods we tested. Prototype heads added 2 to 3 points.

Two: where you train matters more than how much. LoRA, with 1.4% of the weights, matches training a third of the network.

Three: failed ideas were useful. Each one ruled out a wrong explanation.

Four, our limits. We ran each setting once, so differences under 0.3 points are noise. And the efficient methods shared one untuned learning rate.

Five: after the deadline, larger images added 4.4 points on our holdout. Next, we would trade some epochs for larger images, within the budget.

Thank you. We are happy to take your questions.

Notes for difficult terms

- **Noise:** the small random ups and downs in a score. With 20,000 holdout photos, it is about 0.3 points.
- The last two sentences lead into slide 15 (Thank you).